

Name : Preity Mestri TE - B3 batch Roll No. 13256 Practical No:3 Statistical Summary .
Types of Variables.

```
In [3]: import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from scipy import stats
```

```
In [5]: df=pd.read_csv("C:/Users/Preity/Downloads/Placement_Data_Full_Class.csv")
```

```
In [6]: df
```

```
Out[6]:
```

	sl_no	gender	ssc_p	ssc_b	hsc_p	hsc_b	hsc_s	degree_p	degree_t
0	1	M	67.00	Others	91.00	Others	Commerce	58.00	Sci&Tech
1	2	M	79.33	Central	78.33	Others	Science	77.48	Sci&Tech
2	3	M	65.00	Central	68.00	Central	Arts	64.00	Comm&Mgmt
3	4	M	56.00	Central	52.00	Central	Science	52.00	Sci&Tech
4	5	M	85.80	Central	73.60	Central	Commerce	73.30	Comm&Mgmt
...	...	...	...	...	...	...	...	...	...
210	211	M	80.60	Others	82.00	Others	Commerce	77.60	Comm&Mgmt
211	212	M	58.00	Others	60.00	Others	Science	72.00	Sci&Tech
212	213	M	67.00	Others	67.00	Others	Commerce	73.00	Comm&Mgmt
213	214	F	74.00	Others	66.00	Others	Commerce	58.00	Comm&Mgmt
214	215	M	62.00	Central	58.00	Others	Science	53.00	Comm&Mgmt

215 rows × 10 columns



```
In [9]: df.info()
```

```

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 215 entries, 0 to 214
Data columns (total 15 columns):
#   Column                Non-Null Count  Dtype
---  -
0   sl_no                 215 non-null   int64
1   gender                215 non-null   object
2   ssc_p                 215 non-null   float64
3   ssc_b                 215 non-null   object
4   hsc_p                 215 non-null   float64
5   hsc_b                 215 non-null   object
6   hsc_s                 215 non-null   object
7   degree_p              215 non-null   float64
8   degree_t              215 non-null   object
9   workex                215 non-null   object
10  etest_p               215 non-null   float64
11  specialisation         215 non-null   object
12  mba_p                 215 non-null   float64
13  status                 215 non-null   object
14  salary                148 non-null   float64
dtypes: float64(6), int64(1), object(8)
memory usage: 25.3+ KB

```

```
In [10]: df.shape
```

```
Out[10]: (215, 15)
```

```
In [11]: df.describe
```

```

Out[11]: <bound method NDFrame.describe of
sc_b      hsc_s  degree_p \
0         1         M  67.00  Others  91.00  Others  Commerce  58.00
1         2         M  79.33  Central 78.33  Others  Science   77.48
2         3         M  65.00  Central 68.00  Central  Arts     64.00
3         4         M  56.00  Central 52.00  Central  Science  52.00
4         5         M  85.80  Central 73.60  Central  Commerce 73.30
..      ...      ...      ...      ...      ...      ...      ...
210      211        M  80.60  Others  82.00  Others  Commerce  77.60
211      212        M  58.00  Others  60.00  Others  Science   72.00
212      213        M  67.00  Others  67.00  Others  Commerce  73.00
213      214        F  74.00  Others  66.00  Others  Commerce  58.00
214      215        M  62.00  Central 58.00  Others  Science   53.00

        degree_t workex  etest_p specialisation  mba_p      status      salary
0      Sci&Tech     No    55.0         Mkt&HR   58.80      Placed  270000.0
1      Sci&Tech    Yes    86.5         Mkt&Fin   66.28      Placed  200000.0
2  Comm&Mgmt     No    75.0         Mkt&Fin   57.80      Placed  250000.0
3      Sci&Tech     No    66.0         Mkt&HR   59.43  Not Placed         NaN
4  Comm&Mgmt     No    96.8         Mkt&Fin   55.50      Placed  425000.0
..      ...      ...      ...      ...      ...      ...      ...
210  Comm&Mgmt     No    91.0         Mkt&Fin   74.49      Placed  400000.0
211  Sci&Tech     No    74.0         Mkt&Fin   53.62      Placed  275000.0
212  Comm&Mgmt    Yes    59.0         Mkt&Fin   69.72      Placed  295000.0
213  Comm&Mgmt     No    70.0         Mkt&HR   60.23      Placed  204000.0
214  Comm&Mgmt     No    89.0         Mkt&HR   60.22  Not Placed         NaN

[215 rows x 15 columns]>

```

```
In [12]: df.isnull().sum()
```

```
Out[12]: sl_no          0
gender          0
ssc_p          0
ssc_b          0
hsc_p          0
hsc_b          0
hsc_s          0
degree_p       0
degree_t       0
workex         0
etest_p       0
specialisation 0
mba_p          0
status         0
salary         67
dtype: int64
```

```
In [13]: df['salary'].mean()
```

```
Out[13]: np.float64(288655.4054054054)
```

```
In [14]: df['ssc_p'].median()
```

```
Out[14]: 67.0
```

```
In [15]: df['salary'] = df['salary'].fillna(df['salary'].mean())
```

```
In [16]: df.head()
```

```
Out[16]:
```

	sl_no	gender	ssc_p	ssc_b	hsc_p	hsc_b	hsc_s	degree_p	degree_t	w
0	1	M	67.00	Others	91.00	Others	Commerce	58.00	Sci&Tech	
1	2	M	79.33	Central	78.33	Others	Science	77.48	Sci&Tech	
2	3	M	65.00	Central	68.00	Central	Arts	64.00	Comm&Mgmt	
3	4	M	56.00	Central	52.00	Central	Science	52.00	Sci&Tech	
4	5	M	85.80	Central	73.60	Central	Commerce	73.30	Comm&Mgmt	

```
In [17]: from sklearn.preprocessing import LabelEncoder
le = LabelEncoder()
df['gender'] = le.fit_transform(df['gender'])
```

```
In [18]: df['status'] = le.fit_transform(df['status'])
```

```
In [19]: df['workex'] = le.fit_transform(df['workex'])
```

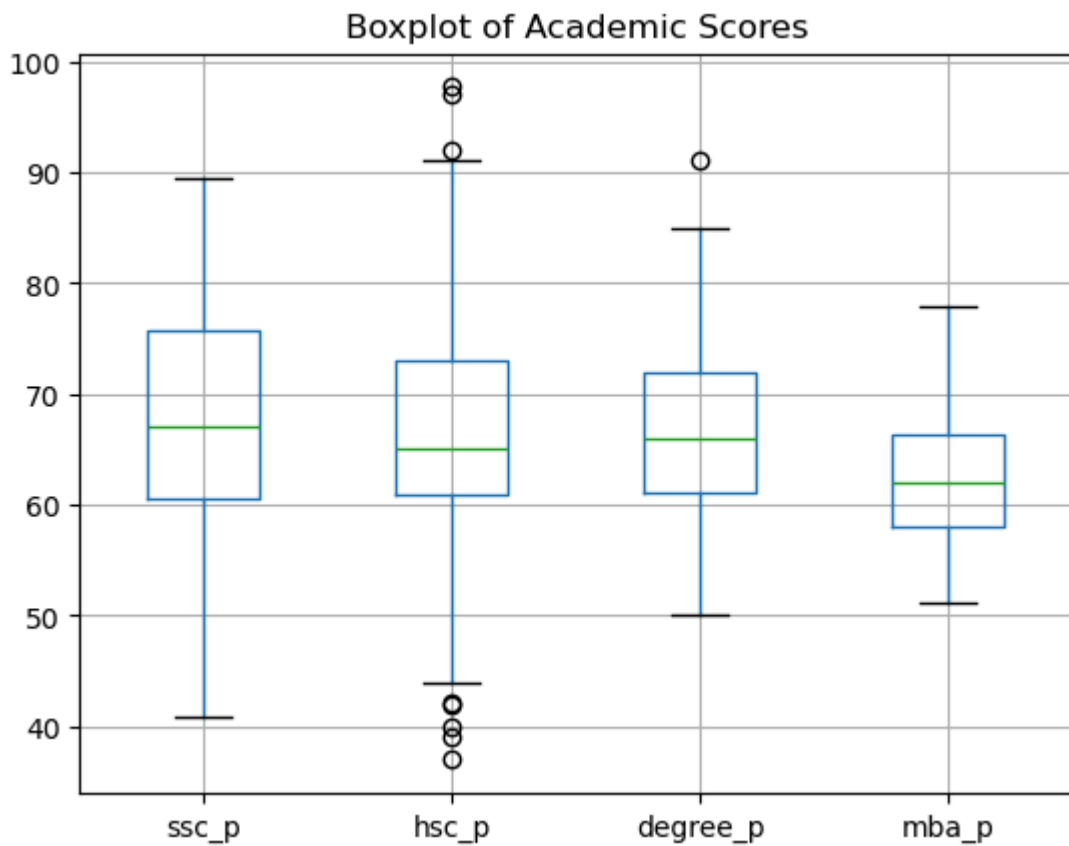
```
In [20]: df['specialisation'] = le.fit_transform(df['specialisation'])
df.head()
```

Out[20]:

	sl_no	gender	ssc_p	ssc_b	hsc_p	hsc_b	hsc_s	degree_p	degree_t	w
0	1	1	67.00	Others	91.00	Others	Commerce	58.00	Sci&Tech	
1	2	1	79.33	Central	78.33	Others	Science	77.48	Sci&Tech	
2	3	1	65.00	Central	68.00	Central	Arts	64.00	Comm&Mgmt	
3	4	1	56.00	Central	52.00	Central	Science	52.00	Sci&Tech	
4	5	1	85.80	Central	73.60	Central	Commerce	73.30	Comm&Mgmt	

In [21]:

```
df.boxplot(column=['ssc_p','hsc_p','degree_p','mba_p'])
plt.title("Boxplot of Academic Scores")
plt.show()
```



In [22]:

```
z = np.abs(stats.zscore(df['mba_p']))
df[z > 3]
```

Out[22]:

sl_no	gender	ssc_p	ssc_b	hsc_p	hsc_b	hsc_s	degree_p	degree_t	workex	etest_p
-------	--------	-------	-------	-------	-------	-------	----------	----------	--------	---------

In [23]:

```
Q1 = df['hsc_p'].quantile(0.25)
Q3 = df['hsc_p'].quantile(0.75)
IQR = Q3 - Q1
lower = Q1 - 1.5 * IQR
upper = Q3 + 1.5 * IQR
print("Lower Bound:", lower)
print("Upper Bound:", upper)
```

Lower Bound: 42.75
Upper Bound: 91.15

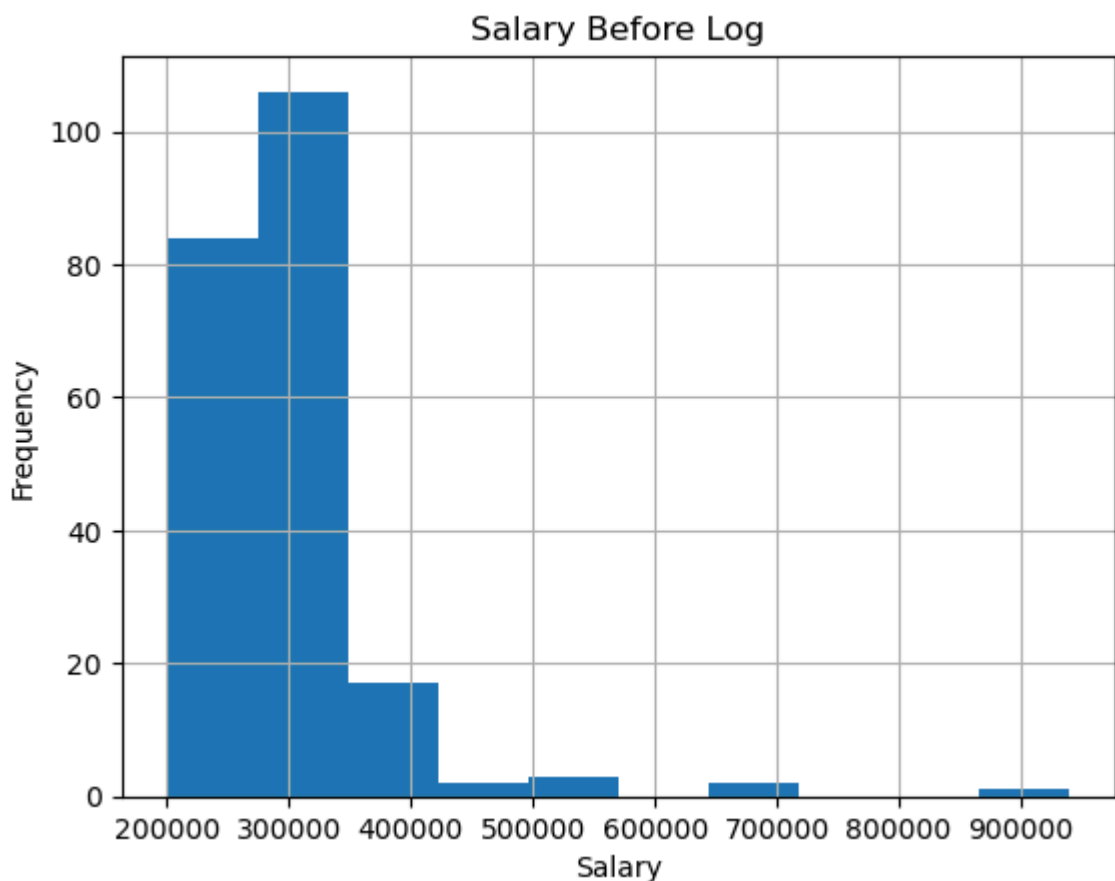
```
In [24]: df[(df['hsc_p'] < lower) | (df['hsc_p'] > upper)]
```

```
Out[24]:
```

	sl_no	gender	ssc_p	ssc_b	hsc_p	hsc_b	hsc_s	degree_p	degree_t
24	25	1	76.50	Others	97.70	Others	Science	78.86	Sci&Tech
42	43	1	49.00	Others	39.00	Central	Science	65.00	Others
49	50	0	50.00	Others	37.00	Others	Arts	52.00	Others
120	121	1	58.00	Others	40.00	Others	Science	59.00	Comm&Mgmt
134	135	0	77.44	Central	92.00	Others	Commerce	72.00	Comm&Mgmt
169	170	1	59.96	Others	42.16	Others	Science	61.26	Sci&Tech
177	178	0	73.00	Central	97.00	Others	Commerce	79.00	Comm&Mgmt
206	207	1	41.00	Central	42.00	Central	Science	60.00	Comm&Mgmt

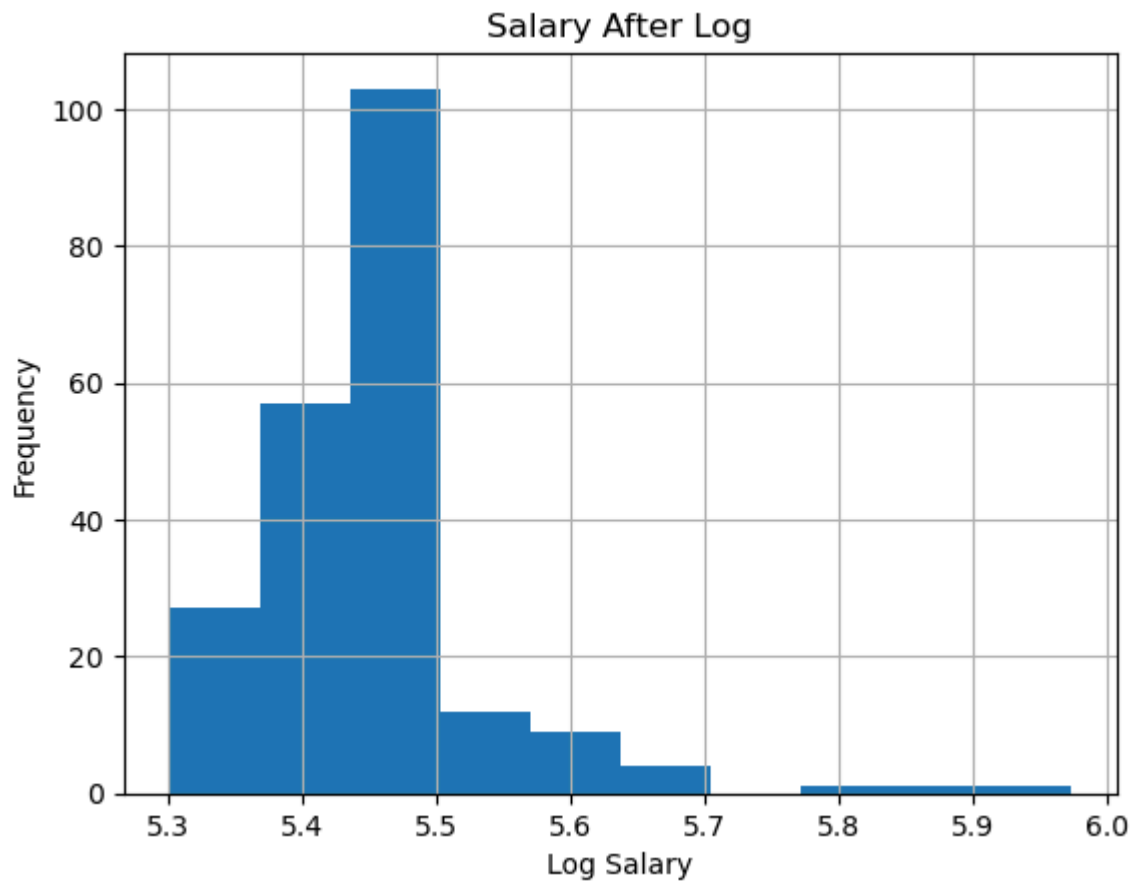
```
In [25]: median = df['hsc_p'].median()  
df['hsc_p'] = np.where((df['hsc_p'] < lower) | (df['hsc_p'] > upper), median, df['
```

```
In [26]: df['salary'].hist()  
plt.title("Salary Before Log")  
plt.xlabel("Salary")  
plt.ylabel("Frequency")  
plt.show()  
df['salary'].skew()
```



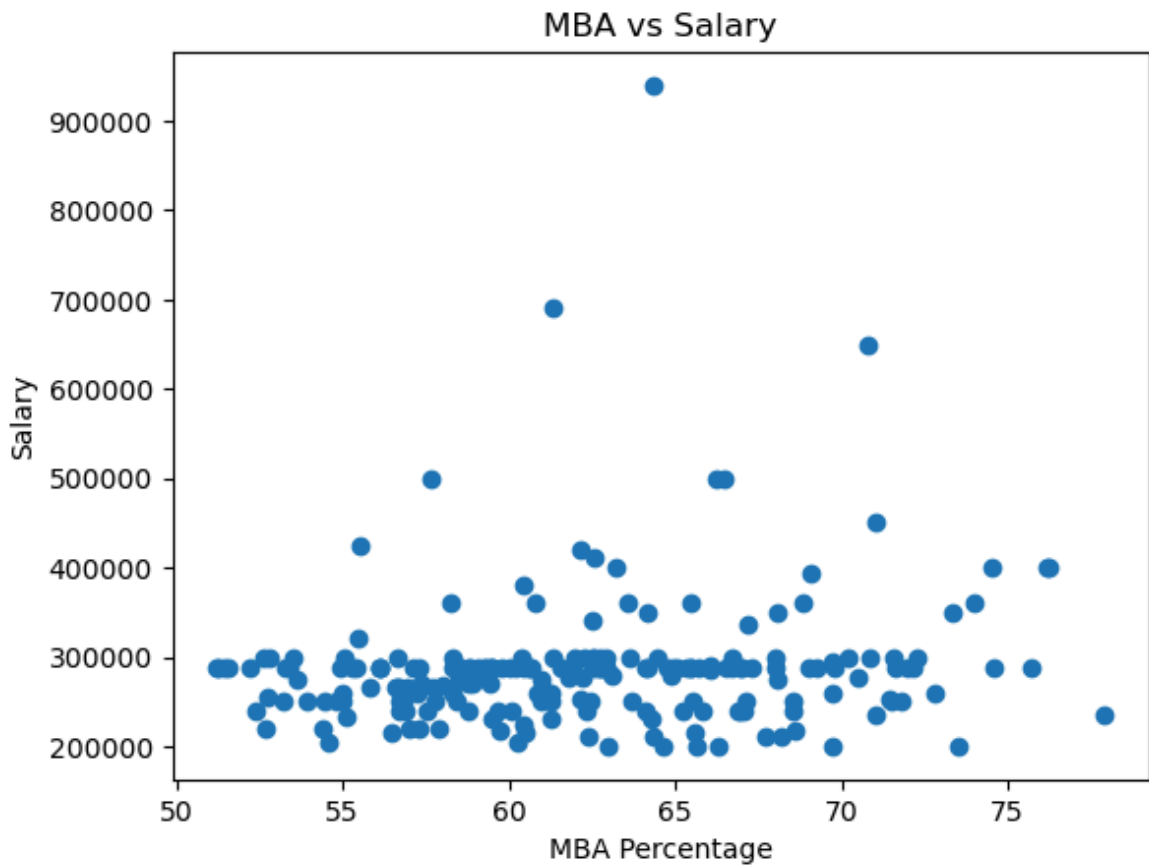
Out[26]: np.float64(4.288798850070048)

```
In [27]: df['log_salary'] = np.log10(df['salary'])
df['log_salary'].hist()
plt.title("Salary After Log")
plt.xlabel("Log Salary")
plt.ylabel("Frequency")
plt.show()
df['log_salary'].skew()
```



Out[27]: np.float64(1.9500125921488516)

```
In [28]: plt.scatter(df['mba_p'], df['salary'])
plt.xlabel("MBA Percentage")
plt.ylabel("Salary")
plt.title("MBA vs Salary")
plt.show()
```



```
In [29]: mad = (df['salary'] - df['salary'].mean()).abs().mean()
print("Mean Absolute Deviation:", mad)
```

Mean Absolute Deviation: 39441.1062225016

```
In [30]: variance = df['salary'].var()
print("Variance:", variance)
```

Variance: 5999726288.204086

```
In [31]: std_dev = df['salary'].std()
print("Standard Deviation:", std_dev)
```

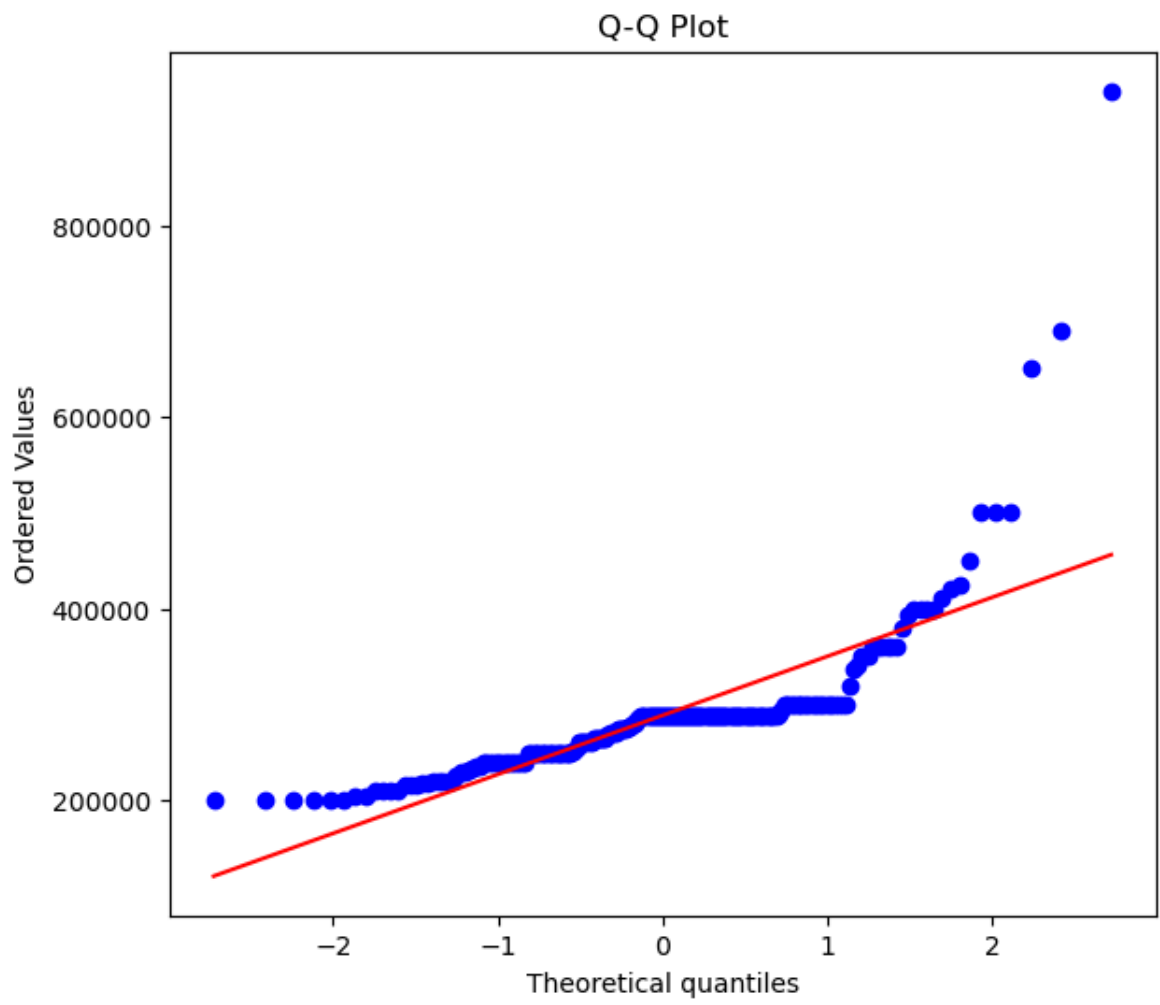
Standard Deviation: 77457.90010195272

```
In [32]: data_range = df['salary'].max() - df['salary'].min()
print("Range:", data_range)
```

Range: 740000.0

```
In [33]: def dgraph(df, variable):
plt.figure(figsize=(15,6))
plt.subplot(1, 2, 1)
stats.probplot(df[variable], dist='norm', plot=plt)
plt.title("Q-Q Plot")
plt.show()
```

```
In [34]: dgraph(df, 'salary')
```



In []: